

第三章

80C51单片微机的指令系统

3.1 80C51指令系统概述

- 指令系统

属于CISC 指令集，指令长度不一

有255种操作码 (00H至FFH，除A5H外)

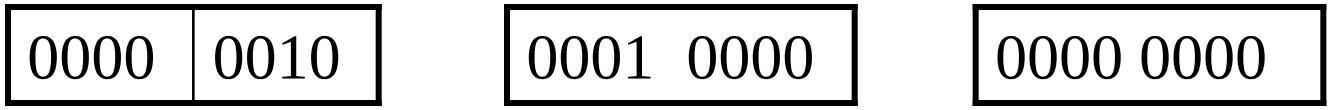
42种助记符（汇编语言），代表了33种功能

共有111种指令

指令由操作码和操作数组成：



LJMP 1000H

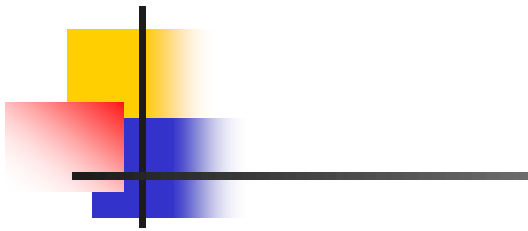


□ 寻址方式：

定义：在指令中给出的寻找操作数或操作数所在地址的方法

80C5l系列单片微机指令系统中共有以下7种寻址方式：

- **寄存器寻址**
- **直接寻址**
- **立即寻址**
- **变址寻址**
- **寄存器间接寻址**
- **相对寻址**
- **位寻址**



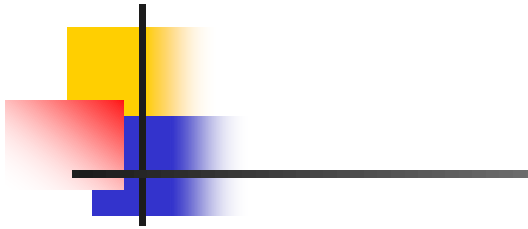
➢ 寄存器寻址：

- 指在指令中将指定寄存器的内容作为操作数。
- 寄存器寻址方式中，用符号名称来表示寄存器。

  例如：指令INC R0
- 寄存器寻址方式的寻址范围包括：

  四个寄存器组，32个字节（通过RS0, RS1选择）

  部分专用寄存器：A 、AB 、DPTR



➢ 直接寻址：

- 指在指令中直接给出操作数单元的地址。

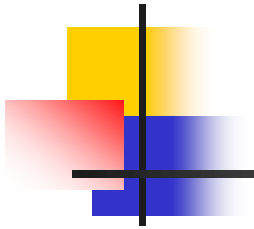
  例如：指令 MOV A，3AH

- 寻址范围：限于片内RAM,包括

  低128单元 （在指令中直接以单元地址形式给出）

  特殊功能寄存器（访问特殊功能寄存器的唯一方法）

虽然特殊功能寄存器可以使用符号标志，但在指令代码中还是按地址进行编码的



➢ 立即寻址：

- 指在指令中中直接给出操作数。

** 为了与直接寻址指令中的直接地址相区别，在立即数前面必需加上前缀“#”。

例如：MOV DPTR，＃1234H

MOV A，#3AH

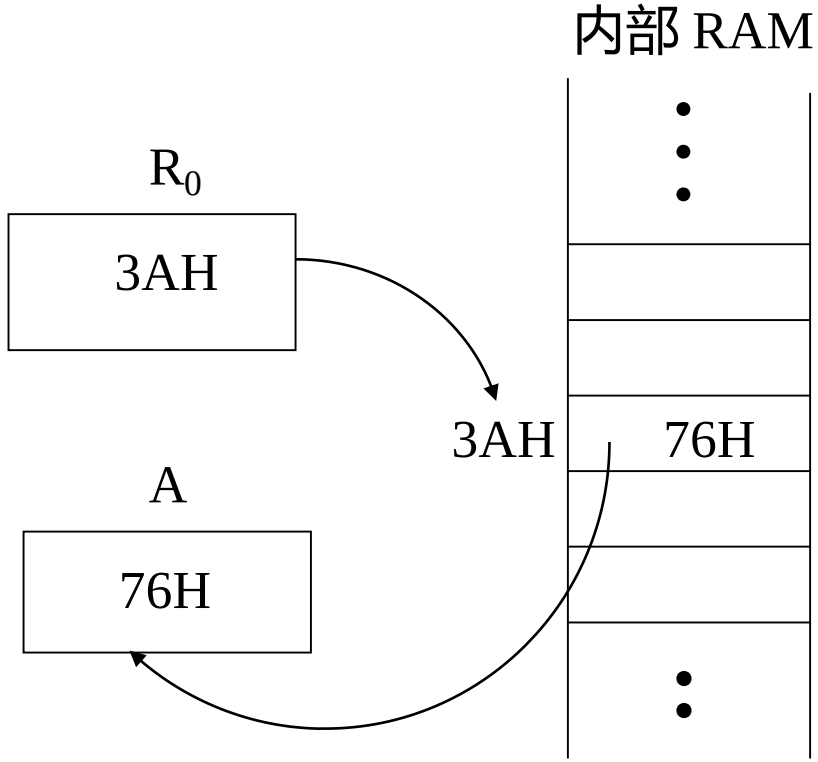
➢ 寄存器间接寻址：

- 指在指令中给出的寄存器内容是操作数的地址，从该地址中取出的才是操作数本身。

** 在寄存器的名称前面加前缀“@”。

例如：ANL　A，@R1

　　MOV　A，@R0



寄存器间接寻址示意图

- 寄存器间接寻址的寻址范围：

(1) **片内RAM的低128单元(8051)**，只能采用R0或R1为间址寄存器，其形式为@Ri（i＝0，1）。

(2) **片外RAM 的64 KB单元**，使用DPTR作为间址寄存器，其形式为@DPTR。

(3) **片外RAM低256单元**，可使用DPTR，或 R0或R1作为间址寄存器外，

(4) **堆栈操作指令（PUSH和POP）**，以堆栈指针（SP）作间址寄存器的间接寻址方式，访问片内RAM。（第一种的补充）



- 位寻址：

- 对数据位进行操作

  例如：ANL　C，30H

- 寻址范围：

  内部RAM中的位寻址区，128位

  专用寄存器中的可寻址位：11个SFR，83位

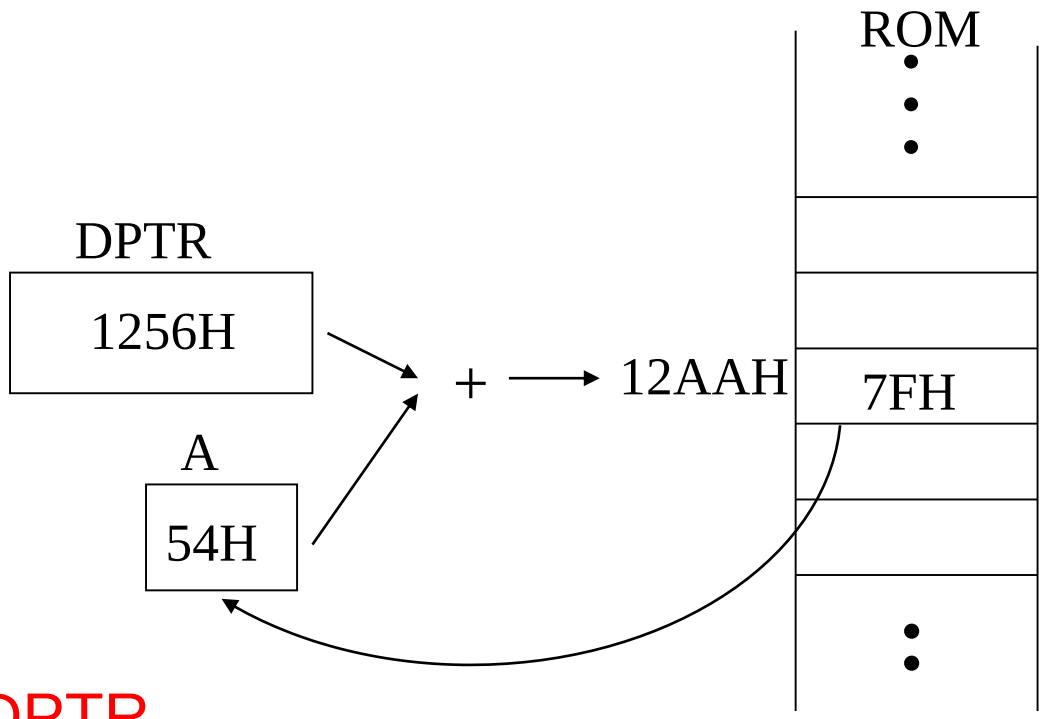
  寻址位的表示方法：

直接使用位地址	D5H
位名称	F0
单元地址加位	D0H.5
专用寄存器符号加位	PSW.5

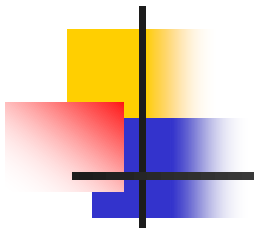
➢ 变址寻址：

- 以DPTR或PC作基址寄存器，累加器A作变址寄存器，以两者内容相加。形成的16位程序存储器地址作为操作数地址。

例如： MOVC A，@A+DPTR



变址寻址示意图



- 变址寻址指令一共三条，还有两条是：

  MOVC　A，　@A+PC

  JMP　　@A+DPTR

- **变址寻址方式是专门针对程序存储器**
- 变址寻址指令都是一字节指令
- **变址寻址一般用于查表操作**

➢ 相对寻址：

- 指在指令中给出的操作数为程序转移的偏移量。

把PC的当前值加上指令中给出的地址偏移量rel，构成了程序转移的目的地址

例如： JNZ rel

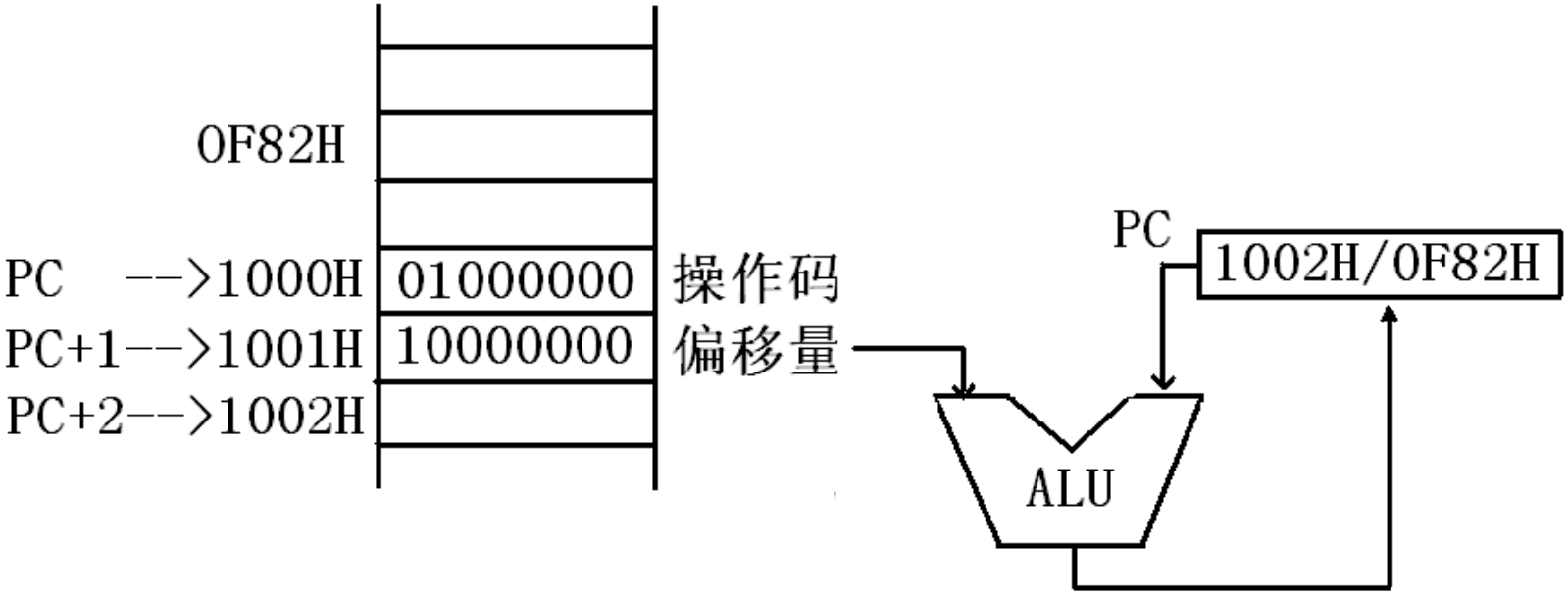
偏移量rel是一个以**补码**形式表示的带符号的8位二进制数

目的地址=转移指令地址+转移指令字节数+rel

或 **目的地址=转移指令的下一条指令地址+rel**

- 128 ~ 127

例如： JC 80H


0F82H
PC --->1000H
PC+1--->1001H
PC+2--->1002H
01000000
10000000
操作码
偏移量
PC
1002H/0F82H
ALU

- 寻址方式总结：

- 上述7种寻址方式针对源操作数
- 目的操作数寻址方式简单，只有4种方式

直接寻址，寄存器寻址，寄存器间接寻址，位寻址

指令格式：

<指令代码>　<目的操作数>，<源操作数>

MOV　　A,　　#3AH

寻址方式小结

序号	寻址方式	使用的变量	寻址空间
1	寄存器寻址	R0~R7、A、B、DPTR	
2	直接寻址		片内RAM低128B 专用寄存器
3	寄存器间接寻址	@R0、 @R1、SP	片内RAM
		@R0、 @R1、 @DPTR	片外RAM
4	立即寻址		程序存储器
5	位寻址		片内RAM的位寻址区 可以位寻址的专用寄存器位
6	变址寻址	@A+DPTR、 @A+PC	程序存储器
7	相对寻址	PC+偏移量	程序存储器

□ 指令格式：

单字节指令：

操作码 地址码

INC DPTR

1010	0011

双字节指令：

操作码 地址码	数据地址码

MOV A, #data

0111	0100

立即数

三字节指令：

操作码	数据地址码	数据地址码

ANL direct, #data

0101	0011	直接地址	立即数

□ 指令系统：

有255种操作码 (00H至FFH，除A5H外)

42种助记符（汇编语言），代表了33种功能

共有111种指令，5大类

⑴ 数据传送类指令（28条）；

⑵ 算术运算类指令（24条）；

⑶ 逻辑运算类指令（25条）；

⑷ 控制转移类指令(17 条)；

⑸ 布尔操作（位）类指令（17条）。

指令系统中使用的符号说明：

Rn ——当前寄存器组的8个通用寄存器R0~R7，所以n=0~7。

Ri ——可用作间接寻址的寄存器，只能是R0、R1两个寄存器，所以 i=0，1。

direct ——内部RAM的8位地址。既可以是内部RAM的低128个单元地址，也可以是专用寄存器的单元地址或符号，因此在指令中direct表示直接寻址方式。



#data ——8位立即数。

data16 ——16位立即数。

addr16 ——16位目的地址，只限于在LCALL和LJMP指令中使用。

addr11 ——11位目的地址，只限于在ACALL和 AJMP指令中使用。

rel ——相对转移指令中的偏移量，为8位带符号补码数。

DPTR ——数据指针

bit ——内部RAM（包括专用寄存器）中的直接寻址位。

A ——累加器。（直接寻址方式的累加器表示为ACC）。

B ——B寄存器。

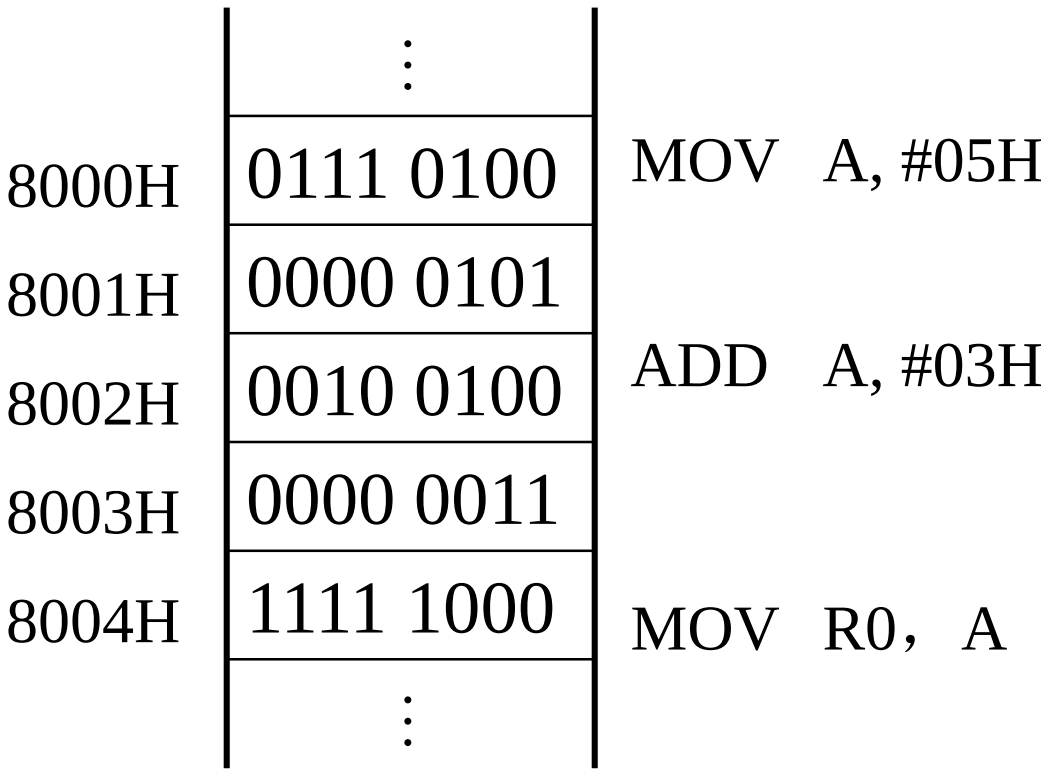
C ——进位标志位，它是布尔处理机的累加器，也称之为累加位。

□ 单片机执行指令的过程

取指、译码，执行

5+3 结果存入R0

```
MOV   A，#05H
ADD   A，#03H
MOV   R0，A
```





指令名称	助记符形式	指令代码		注 释
立即数传送	MOV A，#05H	01110100 00000101	74 05	一个加数送累加器A
加立即数	ADD A，#03H	00100100 00000011	24 03	累加器A内容（05）与立即数（03）相加，其和在A中
数据传送	MOV R0，A	11111000	F8	相加结果送寄存器R0

第一步：取指、译码操作

①PC的内容作为地址送出

②PC的内容自动加1，以指向下一存储单元。

③送出的地址选中8000H单元。

④单片机发出/PSEN命令

⑤在/PSEN命令控制下，将选中的8000H单元内容（指令操作码）读出。

⑥读出的操作码送指令寄存器IR。

⑦经指令译码器译码，发出执行本指令所需的各种控制信号。

第二步：执行指令

①PC的内容作为地址送出

②PC的内容自动加1，以指向下一存储单元。

③送出的地址选中8001H单元。

④单片机发出/PSEN命令

⑤在/PSEN命令控制下，将选中的8001H单元内容（立即数05H）读出。

⑥读出的立即数经数据线传送给累加器A。

3.2 数据传输类指令

- 概述：

通用格式：

MOV <目的操作数>， <源操作数>

其他指令助记符：

MOVX: 外部RAM数据

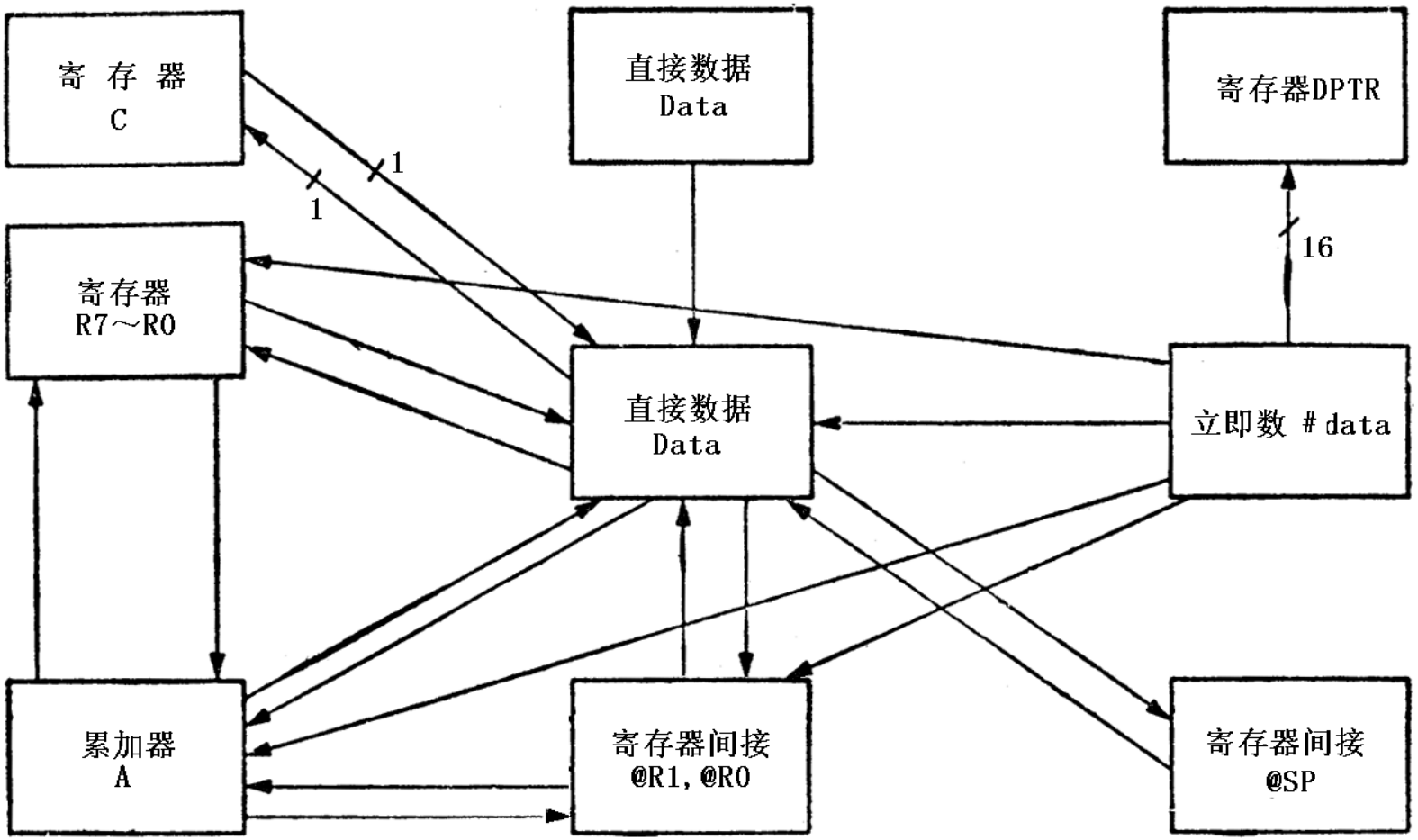
MOVC：程序存储器数据

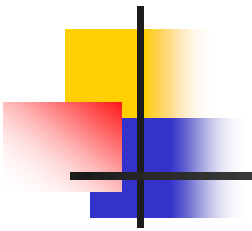
XCH/XCHD/SWAP：数据交换

PUSH/POP：堆栈操作

数据传送指令的一般操作是把源操作数传送到目的操作数，指令执行后源操作数不变，目的操作数被修改为源操作数。若要求进行数据传送时，目的操作数不变，则可以用交换指令。

数据传送方向及相互关系





□ 内部RAM数据传送指令组：16条

指令格式：MOV ＜目的字节＞，＜源字节＞

特点：复制性质

1）以累加器A为目的操作数的指令组

MOV A，Rn ； (A) ←(Rn) R0-R7

MOV A，direct ； (A)←(direct) 00－0FFH

MOV A，@Ri ； (A)←((Ri)) 00－0FFH

MOV A，＃data ； (A)←＃data #00 - #0FFH

绝大部分传送操作均需通过A进行的

2）以工作寄存器Rn为目的操作数的指令组

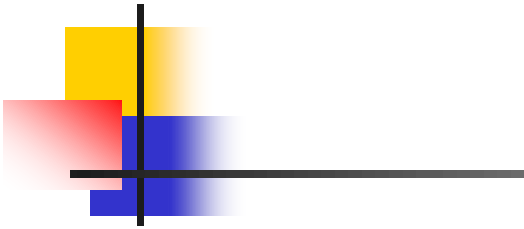
```
MOV     Rn，A          ；(Rn)←(A)
MOV     Rn，direct     ；(Rn)←（direct）
MOV     Rn，＃data     ；(Rn)←＃data
```

direct：RAM 0～127个地址单元和 128～255地址的特殊功能寄存器

3）以直接地址direct（直接寻址）为目的操作数的指令组

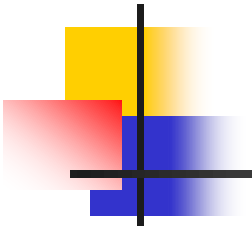
```
MOV     direct，A       ；(direct)←(A)
MOV     direct，Rn      ；(direct)←(Rn)
MOV     direct，direct  ；(direct)←（direct）
MOV     direct，@Ri     ；(direct)←((Ri))
MOV     direct，#data   ；(direct)←#data
```

- **源操作数有寄存器寻址、直接寻址、寄存器间接寻址和立即寻址等寻址方式。**
- **direct：RAM 0～127个地址单元和 128～255地址的特殊功能寄存器**



4）以间接寻址寄存器Ri为目的操作数的指令组

```
MOV     @Ri，A              ；((Ri))←(A)
MOV     @Ri，direct         ；((Ri))←（direct）
MOV     @Ri，＃data    ；((Ri))←＃data
```



5）16位数据传输指令

MOV　DPTR，＃data16；（DPTR）←＃data16

- DPTR是寄存器寻址的，
- 与MOV A, #data是属于一类指令，只是数据是16位的
- 唯一的一条16位数据传输指令

□ 外部RAM数据传送指令组：4条

1）使用DPTR进行间接寻址

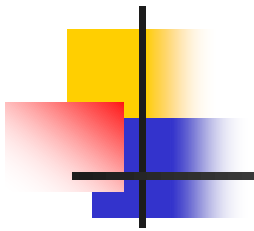
MOVX　　A，@DPTR　；A←（（DPTR））

MOVX　　@DPTR，A　；（DPTR）←（A）

2）使用Ri进行间接寻址

MOVX　　A，@Ri　　；A←（（Ri））

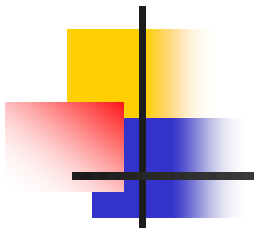
MOVX　　@Ri，A　　；（Ri）←（A）



特点：

•外部RAM数据传送，只能通过累加器A进行

• 指令助记符MOVX,“X”代表外部



举例：某应用系统外扩了8K字节RAM，要求把内部RAM的20H单元内容发送到外部RAM的800H单元中。

```
MOV    DPTR,#800H  ；外部数据存储器地址指针
MOV    R0,#20H     ；内部数据存储器地址指针
MOV    A,@R0       ；取内部数据存储器20H单元内容
MOVX   @DPTR,A     ；送外部数据存储器800H单元
```

□ 读程序存储器数据指令：2条

共2条，通常用于查表

MOVC　A，@A+PC

MOVC　A，@A+DPTR

- DPTR为基址寄存器时，允许数表存放在程序存储器的任意单元，称为远程查表，编程比较直观；
- 而PC为基址寄存器时，数表只能放在该指令单元往下的256个单元中，称为近程查表。编程时需计算A值与数表首址的偏移量。

例3：求平方数(远程查表法)

```
        MOV  DPTR, #TABLE        ；指向表首址
        MOVC  A, @A+DPTR         ；查表得到平方数
        MOV  20H,A               ；存平方数
HERE:   SJMP  HERE
TABLE : DB  00H,01H,04H,09H      ；平方表0²-9²
        DB  16H,25H,36H
        DB  49H,64H,81H
```

例4：求平方数(近程查表)

```
0000H:      ADD  A，#rel                ；修正偏移量，2字节
0002H:      MOVC  A，@A+PC              ；查表得到平方数，1字节
0003H:      MOV  20H，A                 ；存平方数，2字节
 HERE :     SJMP   HERE                 ；2字节
(0005H)
(0007H)
 TABLE :    DB  00H，01H，04H，09H，16H   ；平方表0²-9²
            DB  25H，36H，49H，64H，81H
```

注：rel=TABLE－(查表指令地址＋1)；MOVC指令为单字节。

□数据交换指令：

1）整字节交换指令

XCH　　A，Rn　　；（A）↔（Rn）

XCH　　A，direct　　；（A）↔（direct）

XCH　　A，@Ri　　；（A）↔（（Ri））

特点：交换而非复制性质

例：设（R0）＝30H，（A）＝3FH，片内（30H）＝BBH。

执行指令　XCH A，@R0

执行结果　（A）=BBH，（30H）＝3FH。

2）半字节交换指令

XCHD A，@Ri ；$(A)_{3\sim0} \leftrightarrow ((Ri))_{3\sim0}$

将Ri间接寻址单元的低4位内容与累加器A的低4位内容互换，而它们的高4位内容均不变。此指令不影响标志位。

例：设（R0）＝20H，（A）＝36H（00110110B），内部RAM中（20H）＝75H（01110101B）。

执行指令：XCHD，A，@R0

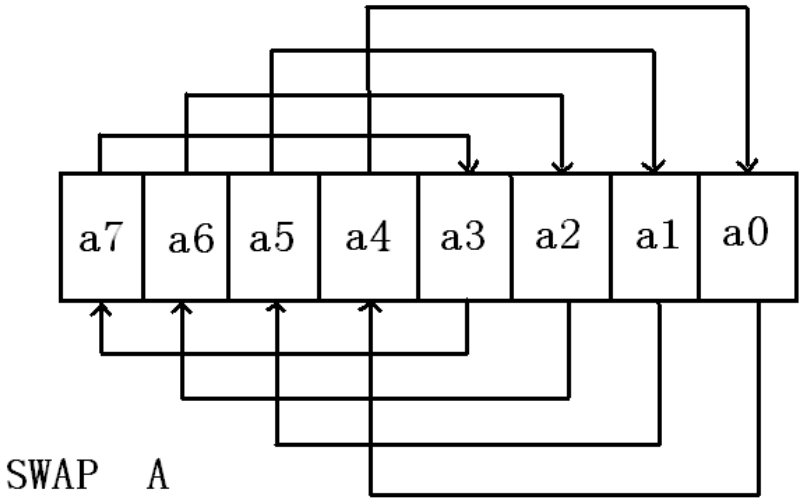
结果为：（20H）＝01110110B＝76H，

（A）＝00110101B＝35H。

3）累加器高低半字节交换指令

SWAP　　A　　；$(A)_{3\sim0} \leftrightarrow (A)_{7\sim4}$

将累加器A的高、低半字节交换，该操作也可看作是四位循环指令。



例：设（A）＝36H（0011 0110B）

执行SWAP A指令，执行结果为（A）＝63H（0110 0011B）。

应用场合：

XCHD和SWAP指令主要用于实现十六进制数或BCD码的数位交换。

□ 堆栈操作指令组：

- 进栈指令：

  内部RAM低128个单元或专用寄存器内容送栈顶单元

  PUSH direct ；SP←(SP)+1，(SP) ←(direct)

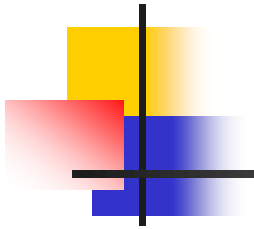
- 出栈指令

  栈顶单元内容送内部RAM低128个单元或专用寄存器

  POP direct ；direct←((SP))，SP ← (SP)–1

本质：

以SP为间址寄存器的间接寻址方式，SP隐含



例：中断响应时(SP) = 30H，DPTR的内容为0123H，执行入栈指令其结果怎样?

PUSH　DPL　　；低8位数据指针寄存器DPL内容入栈

PUSH　DPH　；高8位数据指针寄存器DPH内容入栈。

执行结果:

第一条指令(SP) + 1 = 31H→(SP),(DPL) = 23H→(31H)

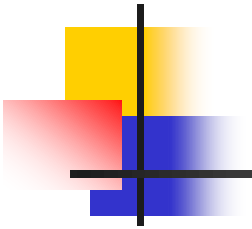
第二条指令(SP) + 1 = 32H→(SP),(DPH) = 01H→(32H)

RAM:　(31H) = 23H，(32H) = 01H，(SP) = 32H

例：设（SP）＝32H，片内RAM的30H~32H单元中的内容分别为20H，23H，01H，执行下列指令的结果怎样?

POP DPH ; ((SP)) = (32H) = 01H→DPH
(SP) - 1 = 32H - 1 = 31H→SP
POP DPL ; ((SP)) = (31H) = 23H→DPL
(SP) - 1 = 31H - 1 = 30H→SP
POP SP ; (SP) - 1 = 2FH→SP
((SP)) = (30H) = 20H→SP

以上第三条指令为特殊情况，先执行（SP）－1＝2FH，后装入由栈顶退出的值，所以执行后（SP）＝20H。



□ 数据传送类指令汇总及说明：

1）同样的数据传送可以使用不同的寻址方式实现

① MOV　　26H，A

② MOV　　R0，#26H

　MOV　　@R0，A

③ MOV　　26H，ACC

④ MOV　　26H，0E0H

⑤ PUSH　　ACC

　POP　　26H

2） A和ACC属不同的寻址方式，指令字节数不同

MOV 26H，A

MOV 26H，ACC

3） 数据传送类指令不影响PSW

一般传送指令传送数据关系

目的操作数	源操作数				
	A	Rn	direct	@Ri	#data
A		√	√	√	√
Rn	√		√		√
direct	√	√	√	√	√
@Ri	√		√		√

例5：检查传送结果

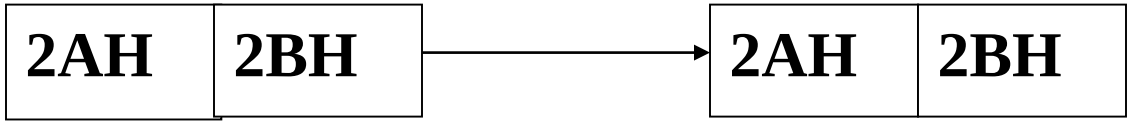
已知内部RAM（10H)=00H，（30H)=40H，(40H)=10H，P1口为11001010B，分析指令执行后各单元内容。

```
MOV  R0，#30H          ；R0=30H
MOV  A，@R0            ；A=40H
MOV  R1，A             ；R1=40H
MOV  B，@R1            ；B=10H
MOV  @R1,P1            ；(40H)=11001010B
MOV  P2，P1            ；P2=11001010B
MOV  10H，#20H         ；(10H)=20H
```

执行结果：**(10H)=20H, (30H)=40H, (40H)=CAH, P1=P2=CAH，A=40H，=10H，R0=30H，R1=40H**

例6： 将4位BCD码倒序。

设内部数据存储器2AH、2BH单元连续存放有4位BCD码数符，试编一程序把4位BCD码数符倒序排列



a3 a2 a1 a0 **a0 a1 a2 a3**



```
        MOV    R0,#2AH
        MOV    R1,#2BH
        MOV    A,@R0      ；2AH单元内容送A
        SWAP   A          ；A的高4位与低4位交换(a2 a3)
        MOV    @R0,A
        MOV    A,@R1      ；2BH单元内容送A
        SWAP   A          ；A的高4位与低4位交换(a0 a1)
        XCH    A, @R0     ；2AH与2BH单元内容交换
        MOV    @R1, A
HERE:   SJMP           HERE
```

例7： 一个有外扩64K RAM的系统，(0000H)=02H, (3000H)=20H,执行以下程序，写出结果

```
MOV P2, #00H;
MOV R0, #00H;
MOV DPTR, #3000H;
MOVX A, @R0
MOV R1, A
MOVX A, @DPTR
MOV R2, A
```

R1=?, R2=?

3.3 算术运算类指令

- 概述：
  - 包括各种算术操作，其中有加、减、乘、除四则运算。
  - 有符号数运算、无符号数运算
  - 二进制十进制调整
  - 除加1和减1指令之外，算术运算结果将影响标志位 OV, CY, AC



1.加法指令组

```
ADD     A，Rn       ；A←（A）+（Rn）
ADD     A，direct   ；A←（A）+（direct）
ADD     A，@Ri      ；A←（A）+（（Ri））
ADD     A，#data    ；A←（A）+ data
```

- 加法运算影响PSW位的状态。如果位3有进位，则辅助进位标志AC置1，否则AC清0；如果位7有进位，则进位标志CY置1，否则CY清0。

- 两个带符号数相加，还有溢出的问题。如运算结果使溢出标志OV置1，则表示有溢出出现。

  对于带符号数运算,当和的第7位与第6位中有一位进位而另一位不产生进位时，溢出标志OV置位,否则为0。（OV）＝1表示两个正数相加,和为负数；或两个负数相加而和为正数的错误结果。

例如：

（A）=C2H，（R0）=A9H

执行 ADD A，R0 指令

```
  1 1 0 0 0 0 1 0
+ 1 0 1 0 1 0 0 1
-----------------
1← 0 1 1 0 1 0 1 1
```

运算结果 （A）=6BH，（AC）=0，（CY）=1，（OV）=1。

若C2H和A9H是两个无符号数，则连进位在内的结果为16BH，运算是正确的。

若C2H和A9H是两个带符号数，则由于有溢出而表明结果是错误的，因为两个负数相加不可能得到正数的和。



2.带进位加法指令组

ADDC A，Rn　　；A←（A）+（Rn）+（CY）

ADDC A，direct　　；A←（A）+（direct）+（CY）

ADDC A，@Ri　　；A←（A）+（（Ri））+（CY）

ADDC A，#data　　；A←（A）+ data +（CY）

这组指令常用于多字节数相加，实现8的倍数位(如16位、32位)数的加法



例：设（A）＝C3H，（R0）＝AAH，（CY）＝1。

执行指令：ADDC A，R0

```
  1 1 0 0 0 0 1 1
+ 1 0 1 0 1 0 1 0
+               1   (CY)
-----------------
  0 1 1 0 1 1 1 0
```

执行结果：（A）＝6EH，（CY）＝1，（OV）＝l，
（AC）＝0。

对于带符号数的带进位相加，溢出标志为1，意味着出错，上例为二个负数相加，出现结果为正数的错误。

例：已知(A) = B3H，(R1) = 56H。

执行ADD A,R1指令：

B 3H	1 0 1 1 0 0 1 1
+ 5 6H	+ 0 1 0 1 0 1 1 0
1 0 9H	1 0 0 0 0 1 0 0 1
CY=1	CY=1，OV=0，AC=0

若二个数是无符号数，则B3H + 56H=109H，答案正确。

若二个数是带符号数，则B3H的原码是–77D,56H原码为86D(–77D) + 86D = 09D, 答案也是正确的, 因为OV = 0。



3. 加1指令组

```
INC     A       ; A←（A）+1
INC     Rn      ;  Rn ←  (Rn) +1
INC     direct  ; direct ←  ( direct ) +1
INC     @Ri     ;  ( Ri)  ←  (  (Ri) ) +1
INC     DPTR    ; DPTR ←  ( DPTR ) +1
```

加1操作不影响PSW

例：设（R0）＝7EH，（7EH）＝0FFH．（7FH）＝40H。

执行以下指令：

```
INC    @R0     ；0FFH＋1＝00H  仍存入7EH单元
INC    R0      ；7EH＋1＝7FH   存入（R0）
INC    @ R0    ；40H＋1＝41H   存入（7FH）
```

执行结果为（R0）＝7FH，（7EH）＝00H．（7FH）＝41H

4. 十进制调整指令

DA　　A

- 十进制调整指令是紧接着加法指令用的，不能在其它运算后作十进制调整。
- 被加数和加数是十进制BCD码，执行ADD或ADDC二进制加法指令，再用DA　A作十进制调整后，得到的结果是十进制BCD码。
- BCD（Binary-Coded Decimal）



- 累加器低4位大于9或辅助进位标志位（AC）=1，则进行低4位加6修正：

  A←（A）+06H

- 累加器高4位大于9或进位标志（CY）=1，则进行高4位加6修正：

  A←（A）+60H

- 累加器高4位为9，低4位大于9，则进行高低4位均加6修正：

  A←（A）+66H

例：设（A）＝0101 0110＝56 BCD，（R3）＝ 0110 0111＝67 BCD，（CY）＝1。

执行下述二条指令： ADDC A，R3

DA A

执行 ADDC A，R3 （A） 0 1 0 1 0 1 1 0 (56 BCD)

(R3) 0 1 1 0 0 1 1 1 (67 BCD)

\+ （CY） 1 .

10 1 1 1 1 1 0 (高、低四位均大于9)

再执行DA A 0 1 1 0 0 1 1 0 (加66H操作)

Cy=1 0 0 1 0 0 1 0 0 (124 BCD)

即BCD码数56＋67＋1＝124。经DA A指令校正后，答案正确。

例8： 二个多字节无符号数相加。设有二个四位BCD码分别存在内部RAM的50H、51H和60H、61H单元中，试编写程序，求二个BCD码数之和，结果存入内部40H、41H单元。

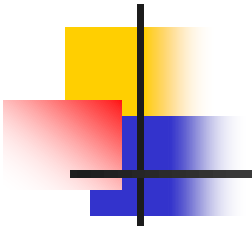
```
MOV  R0,#50H        ；被加数首址
MOV  R1,#60H        ；加数首址
MOV  A,@R0          ；取被加数
ADD  A,@R1          ；与加数相加
DA  A                       ；二—十进制调整
MOV  40H,A          ；存和
INC  R0             ；高位相加
INC  R1
MOV  A,@R0
```

```
        ADDC  A,@R1
        DA  A
        MOV  41H，A
```

例8：对累加器A中压缩BCD码数减1

```
        ADD  A,#99H
        DA   A
HERE:SJMP  HERE
```

说明：累加器A允许的最大BCD码数为99BCD，当对A实行加99BCD码数时，必然形成对BCD码百位数的进位，而剩在A中的内容正是压缩BCD码数减1。如BCD的59H，经DA A的调整后，为58H且CY=1，不考虑进位CY，则BCD码59－1＝58。



5.带借位减法指令组

SUBB A，Rn　　；A←（A）–（Rn）–（CY）

SUBB A，direct ；A←（A）–（direct）–（CY）

SUBB A，@Ri　　；A←（A）–((Ri))–（CY）

SUBB A，#data　；A←（A）–data–（CY）

- 减法运算影响PSW。减法运算时，AC是半借位，CY是借位位。
- 没有不带借位的减法指令
- 带符号数运算时，若OV=1，则结果错误。

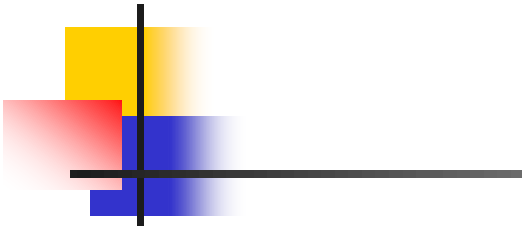
若（A）=C9H （R2）=54H CY=1

执行 SUBB A，R2

（A）=74H （CY）=0

（OV）=1

```
  1 1 0 0 1 0 0 1
– 0 1 0 1 0 1 0 0
-----------------
  0 1 1 1 0 1 0 0
```



6. 减1指令组

```
DEC     A       ; A←（A）–1
DEC     Rn      ; Rn ←（Rn）– 1
DEC     direct  ; direct ←（direct）– 1
DEC     @Ri     ; （Ri）←（（Ri））– 1
```

- **减1操作不影响PSW**
- **DPTR没有减1指令**

7. 乘除指令组

MUL AB

DIV AB

乘法和除法操作都影响PSW

乘法和除法指令都是一字节指令，但执行时间都是4个机器周期。



- 累加器A和寄存器B中的两个8位无符号数相乘，乘积为16位，低8位在A中，高8位在B中。CY=0。当乘积小于、等于FFH时，OV=0；否则OV=1。
- 两个8位无符号数进行除法运算A/B，商在A中，余数在B中。CY=0。
- 除数为0时，OV=1；否则OV=0。

例：设（A）＝50H（80D），（B）＝A0H（160D）。
执行指令：MUL AB
即80×160＝12800＝3200H
执行结果：乘积 3200H（12800），（A）＝00H，（B）＝32H，（OV）＝1，（CY）＝0。

例：设（A）＝0FBH（251D），（B）＝12H（18D）。
执行指令：DIV AB
执行结果：（A）＝0DH（商13），（B）＝11H（余数I7），（OV）＝0，（CY）＝0。

例9： 数的码制转换。把累加器A中无符号二进制整数 (00-FFH) 转换为BCD码 (0～255)，并存入内存30H和31H单元。

```
BINBCD：MOV  B,#100
        DIV  AB                ；A÷10，百位数在A,余数在B
        MOV  30H,A             ；百位数送30H
        MOV  A,B
        MOV  B,#0AH
        DIV  AB       ;余数÷10，十位数在A低四位,个位数在B
        SWAP  A                ；十位数放A的高四位
        ADD  A,B       ；十位数和个位数组合后送31H
        MOV  31H,A
        RET
```

算术运算类指令汇总

算术运算类指令中的操作数，**（一个是A）**

	源 操 作 数					
助记符	B	Rn	direct	@Ri	#data	DPTR
ADD		√	√	√	√	
ADDC		√	√	√	√	
SUBB		√	√	√	√	
MUL	√					
DIV	√					

加1和减1指令中的源操作数对象

助记符	A	Rn	direct	@Ri	DPTR
INC	√	√	√	√	√
DEC	√	√	√	√	

3.4 逻辑运算类指令

概述：

- 各类逻辑运算：与、或、异或、清除、求反、移位等
- 助记符有ANL、ORL、XRL、RL、RLC、RR、RRC、CPL、CLR等九种。
- 只按位进行逻辑运算，结果不影响PSW中标志位。

1. 逻辑与 运算指令组

ANL A，Rn ；（A）←（A）∧（Rn）

ANL A，direct ；（A）←（A）∧（direct）

ANL A，@Ri ；（A）←（A）∧（（Ri））

ANL A，＃data ；（A）←（A）∧＃data

ANL direct，A ；（direct）←（direct）∧（A）

ANL direct，＃data ；（direct）←（drect）∧＃data

按位与，用来做清除或屏蔽某些位

例：设P1=FFH

执行指令：ANL P1，#0F0H

执行结果：

P1 = F0H，P1.7~P1.4位不变，P1.3~P1.0位被清除。

逻辑“与”运算指令用做清除或屏蔽某些位。

0FFH	1	1	1	1	1	1	1	1
0F0H	1	1	1	1	0	0	0	0
AND	1	1	1	1	0	0	0	0

2.逻辑或 运算指令组

ORL	A，Rn	；A←（A）∨（Rn）
ORL	A，direct	；A←（A）∨（direct）
ORL	A，@Ri	；A←（A）∨（(Ri)）
ORL	A，#data	；A←（A）∨ data
ORL	direct，A	；direct←(direct)∨(A)
ORL	direct，#data	；direct←(direct)∨ data

当需要改变一个字节中某几位，而其余位保持不变时，不能使用数据传送指令，只能用逻辑运算指令。

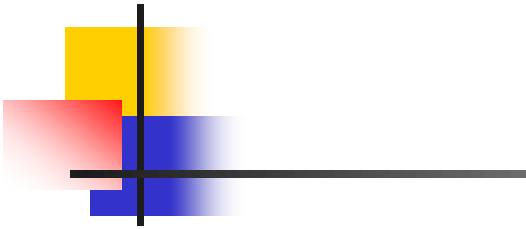
例：设（A）＝0A3H（10100011B），（R0）＝45H（0100 0101B）。

执行指令：ORL A,R0

执行结果：（A）＝E7H（1110 0111B）。

逻辑“或”运算指令用来置位某些位。

0A3H	1	0	1	0	0	0	1	1
45H	0	1	0	0	0	1	0	1
OR	1	1	1	0	0	1	1	1



3.逻辑异或 运算指令组

```
XRL     A，Rn       ；A←（A）⊕（Rn）
XRL     A，direct   ；A←（A）⊕（direct）
XRL     A，@Ri      ；A←（A）⊕（（Ri））
XRL     A，#data    ；A←（A）⊕data
XRL     direct，A   ；direct←(direct)⊕(A)
XRL     direct，#data；direct←(direct)⊕data
```

例：设（A）＝0A3H（1010 0011B），

（R0）＝045H（0100 0l0lB）。

执行指令为 XRL　　A，R0

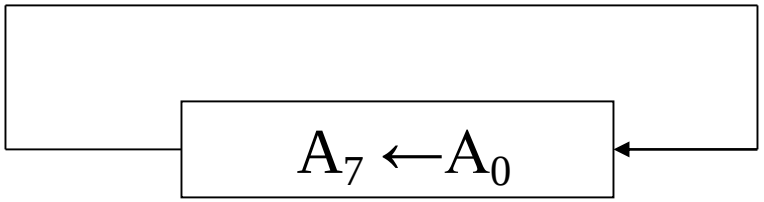
$$
\begin{array}{rr}
 & 1\ 0\ 1\ 0\ 0\ 0\ 1\ 1 \\
\oplus & 0\ 1\ 0\ 0\ 0\ 1\ 0\ 1 \\
\text{结果} & 1\ 1\ 1\ 0\ 0\ 1\ 1\ 0
\end{array}
$$

执行结果为（A）＝　　E6H。

4. 移位指令组

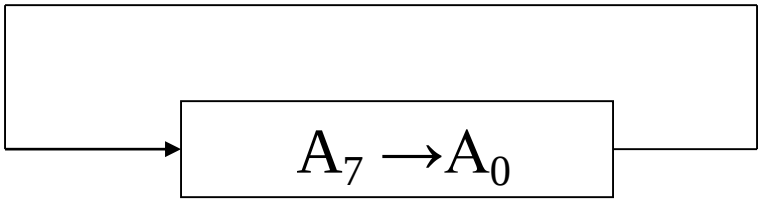
（1）循环左移

RL　　A　　；$A_{n+1} \leftarrow A_n$，$A_0 \leftarrow A_7$



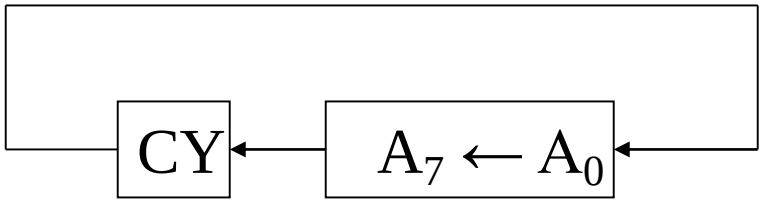
（2）循环右移

RR　　A　　；$A_n \leftarrow A_{n+1}$，$A_7 \leftarrow A_0$



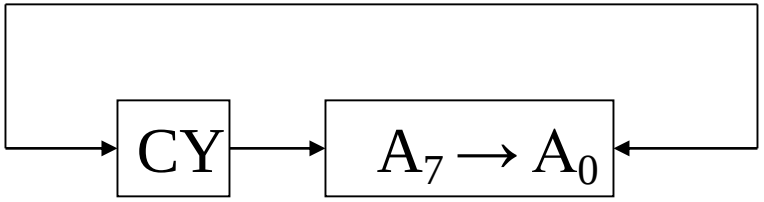
(3) 带进位循环左移

RLC A ； $A_{n+1} \leftarrow A_n$， $CY \leftarrow A_7$， $A_0 \leftarrow CY$



(4) 带进位循环右移

RRC A ； $A_n \leftarrow A_{n+1}$， $A_7 \leftarrow CY$ ， $CY \leftarrow A_0$



例：设（A）＝0A6H（10100110），执行RR A指令，执行结果＝ 53H（01010011B）。

	1	0	1	0	0	1	1	0
RR A	0	1	0	1	0	0	1	1

例：设(A) = 0B4H(10110100B),(CY) = 1，执行RRC A指令，执行结果为：(A) = 0DAH (11011010B),(CY) = 0

	1	1	0	1	1	0	1	1	0
RRC A	0	1	1	0	1	1	0	1	1

CY



5. 累加器清0取反指令组

累加器清0指令：

CLR　　A　　；A←0

累加器按位取反指令：

CPL　　A　　；$A \leftarrow \overline{(A)}$

逻辑运算及移位类指令汇总

双操作数逻辑运算指令
两个操作数之间的配合关系

		第二操作数				
		A	Rn	direct	@Ri	#data
第一操作数	A		√	√	√	√
	direct	√				√

例： 数据的拆分与拼装

要求：从(30H)=$X_7X_6X_5X_4X_3X_2X_1X_0$中取出高5位，从(31H)=$Y_7Y_6Y_5Y_4Y_3Y_2Y_1Y_0$中取出低3位，拼装后存入40H中，(40H)=$Y_2Y_1Y_0X_7X_6X_5X_4X_3$

地址　机器码

```
                        ORG   0000H
0000   E5  30          MOV  A,30H
0002   C4               SWAP  A     ;  X3X2X1X0  X7X6X5X4
0003   23               RL  A ;  X2X1X0  X7X6X5X4X3    左移了5位
0004   F5  40           MOV  40H,A
0006    53  40  1F      ANL  40H,#00011111B ;   000X7X6X5X4X3X2X1
```



```
0009    E5  31              MOV  A,31H
000B    75  F0  20          MOV  B,#20H
000E    A4                  MUL  AB ;  Y2Y1Y0  00000
                                       ; 左移了5位
000F    54  E0              ANL  A,#11100000B; Y2Y1Y0 00000
0011      42 40             ORL  40H,A
0013    21 13       HERE :  AJMP  HERE
```

注：实现左移5位，采用了二种方法，即移位和乘法。

3.5 控制程序转移类指令

概述：

- 共分4类：12种助记符

无条件转移指令： **AJMP、LJMP、SJMP**

条件转移指令： **JZ、JNZ、**

循环转移指令： **CJNE、DJNZ、**

子程序调用和返回指令： **ACALL、LCALL、RET、RETI、**

空操作指令： **NOP**

- 不包括布尔变量控制程序转移指令。

1. 无条件转移指令

指令	机器码		
SJMP　rel	80	rel	
AJMP　addrll	a10 a9 a8 00001	a7---a0	
LJMP　addrl6	02	addr15-8	addr7-0
JMP　@A＋DPTR	73		

- 功能：程序无条件地转移到各自指定的目标地址去执行



1） 短转移指令

SJMP rel ； rel是带符号的8位二进制补码

偏移量的计算：

- **根据偏移量计算转移的目的地址**
- **根据目的地址计算偏移量**

- **根据偏移量计算转移的目的地址**

目的地址=（PC）+2+rel

举例：2100H　SJMP　　7FH；35H是正数

目的地址=2100H+2+7FH=2181H

2100H　SJMP　　80H；80H是负数，–80H

目的地址=2100H+2–80H=2082H

2102H
+ 007FH
2181H

2102H
+ FF80H
2082H

- **根据目的地址计算偏移量**

rel = 目的地址– [（PC）+2]

= 目的地址–跳转指令的下一条指令的地址

如果目的地址小于跳转指令的下一条指令的地址，则计算结果+100H

例：原地跳转指令偏移量计算

2100H　HERE:SJMP　HERE

rel = 2100H - 2102H= -2H →+100H = 0FEH

求补码

2）绝对转移指令

AJMP addr11；PC ← （PC）+2，PC10-0 ←addr11

$a_{10}a_9a_8$0	0 0 0 1
$a_7a_6a_5a_4$	$a_3a_2a_1a_0$

跳转范围： 2 ^ 11 = 2048 2K空间

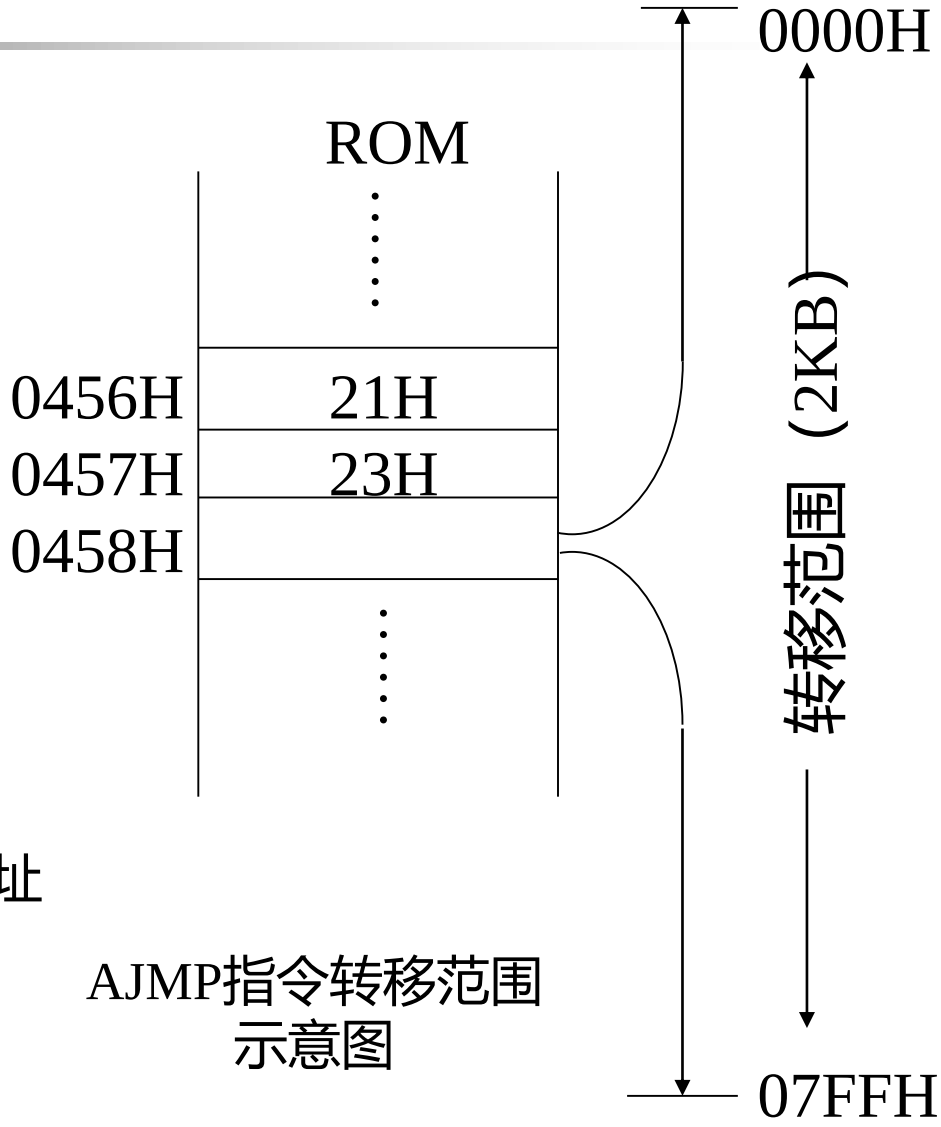
例： 0456H　AJMP　123H

PC ← （PC） + 2，
$PC_{10\sim0}$ ← addr11

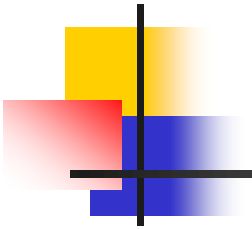
(PC) + 2 = 0458H
目的地址 = 0123H

0000 0100	0101 1000
0000 0001	0010 0011

PC＋2
目标地址



AJMP指令转移范围
示意图



3) 长转移指令

LJMP addr16

例：设（PC）＝0123H，标号ADR所指单元地址为3456H。

执行指令为 LJMP ADR

执行结果为（PC）＝3456H

程序转向3456H单元执行。



4) 间接转移指令

JMP　@A+DPTR

目标地址：PC ← （A）+（DPTR）

- **相加运算不影响累加器A、DPTR的原内容。**
- **若结果大于64 KB，则从程序存储器的零地址往下延续。**

例：设（A）=5，（DPTR）=4567H。

执行指令为 JMP @A + DPTR

执行结果为:(PC) = (A) + (DPTR) = 05H + 4567H = 456CH。

例11：散转程序设计，根据A中的数值实现程序散转

```
                MOV     R1,A            ；(A)×3
                RL      A
                ADD     A,R1
                MOV  DPTR ，  #TABLE ；散转表首地址送DPTR
                JMP     @A+DPTR
TABLE：      LJMP   PM0                    ；转程序PM0
TABLE＋3：   LJMP   PM1                    ；转程序PM1
                ------
 PM0：          -----
```

LJMP是一个三字节指令，因此转移指令入口地址相隔3个字节，A中内容需是3的倍数。

2. 条件转移指令组

1）累加器判零转移指令

JZ　　rel　　；若（A）= 0，则PC ← （PC）+2+rel

　　；若（A）≠0，则PC ← （PC）+2

JNZ　　rel　　；若（A）≠0，则PC ← （PC）+2+rel

　　；若（A） = 0，则PC ← （PC）+2

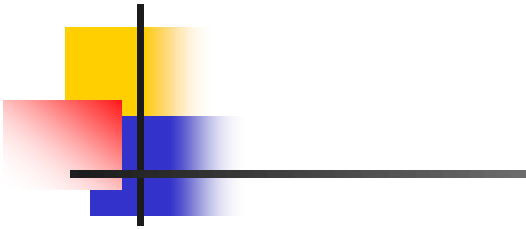
特点：

- **偏移量rel是带符号的8 位数，程序转移的目标地址为指令前128 B或指令后 127 B。**
- **指令执行后不影响任何操作数和标志位**



例：设（A）＝01H　　执行程序

```
    JZ  LABEL1     ；因为（A）≠ 0，程序继续执行
    DEC A          ；  （A）－1＝00H
    JZ  LABEL2     ；因为（A）=00H，程序转向标号
                   ；  LABEL2  指示的地址执行
LABEL1：------
LABEL2：------
```



2）数值比较转移指令

CJNE A，#data，rel ； 累加器内容与立即数不等转移

CJNE A，direct，rel ； 累加器内容与内部RAM单元内容不等转移

CJNE Rn，#data，rel ； 寄存器内容与立即数不等转移

CJNE @Ri，#data，rel； 内部RAM前128单元内容与立即数不等转移

两操作数的比较不影响操作数的值，但影响CY标志，CY的值由无符号数目的操作数–源操作数有无借位决定。



- 指令功能：不等则转移
- 不等，程序转移到(PC) +3再加第三字节带符号 的8位偏移量（rel）指向的目标地址；
- 相等，程序继续执行。
- 程序转移的范围是从（PC） + 3为起始的 + 127 ~ —128的单元地址。

CJNE指令中源操作数与目的操作数的关系

		源操作数	
		direct	#data
目的操作数	A	√	√
	Rn		√
	@Ri		√

3）减1条件转移指令

寄存器减1条件转移指令

DJNZ　Rn，rel　　；Rn ←(Rn) –1

；若(Rn) ≠0，则PC←（PC）+2+rel

；(Rn) = 0，则PC←（PC）+2

•直接寻址单元减1条件转移指令

DJNZ　direct，rel ；direct ←(direct) –1

；若(direct) ≠0，则PC←（PC）+3+rel

；(direct) = 0，则PC←（PC）+3

例：从P1.7引脚输出5个方波

```
        MOV  R2,#10          ；5个方波，10个状态
LOOP：  CPL  P1.7            ；P1.7状态变反
        DJNZ  R2 ,LOOP
```

例11：数据块移动。

将2000H单元开始的一批数据传送到从3000H开始的单元中，数据长度在内部RAM的30H中。

```
        ORG 0000H
        LJMP MAIN
        ORG 0030H
MAIN:   MOV SP, #30H            ; 堆栈初始化
        MOV  DPTR，#2000H       ; 源数据区首址
        PUSH  DPL               ； 源数据区首址压栈保护
        PUSH  DPH
```

```
        MOV  DPTR，#3000H    ；目的数据区首址
        MOV  R6，DPL         ；目的数据区首址存入寄存器
        MOV  R7，DPH
LP：    POP  DPH             ；取数据区地址指针
        POP  DPL
        MOVX  A，@DPTR       ；取源数
        INC  DPTR
        PUSH  DPL
        PUSH  DPH
        MOV  DPL，R6         ；取目的数据区地址指针
        MOV  DPH，R7
        MOVX  @DPTR，A       ；存入目的数据区
        INC  DPTR
        MOV  R6，DPL
        MOV  R7，DPH
        DJNZ  30H，LP        ；若数据块未移完，则继续
```

```
        POP  DPH
        POP  DPL
HERE:   SJMP HERE
```

注意：

因为2000H和3000H都在外部RAM或I/O中，使用地址指针均为DPTR，所以要注意DPTR的保护、恢复

假设两个独立的DPTR，分别为DPTR1和DPTR2

```
MAIN:   MOV SP, #30H          ; 堆栈初始化
        MOV  DPTR1，#2000H    ; 源数据区首址
        MOV  DPTR2，#3000H    ；目的数据区首址
LP:     MOVX  A，@DPTR1       ；取源数
        INC  DPTR1
        MOVX  @DPTR2，A       ；存入目的数据区
        INC  DPTR2
        DJNZ  30H，LP         ；若数据块未移完，则继续
        HERE:  SJMP HERE
```

3.子程序调用与返回指令组

1）绝对调用指令

ACALL　　addr11

$a_{10}a_9a_8$1	0 0 0 1

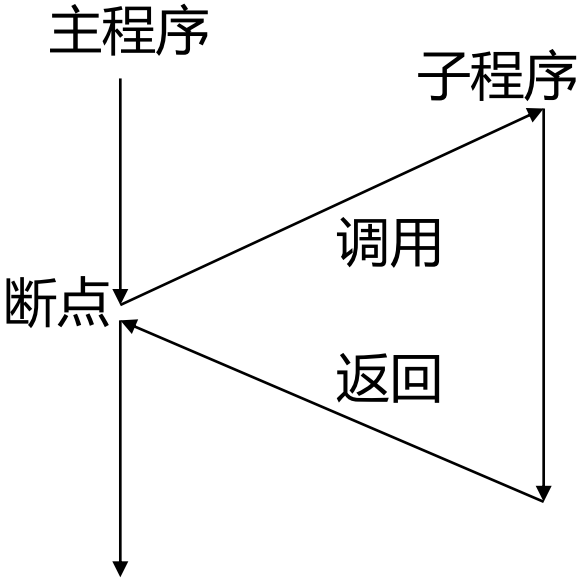
$a_7a_6a_5a_4$	$a_3a_2a_1a_0$

PC←(PC)+2

SP←(SP)+1，(SP)←$(PC)_{7\sim0}$

SP←(SP)+1，(SP)←$(PC)_{15\sim8}$

$PC_{10\sim0}$ ←addr11



子程序调用示意图

例：设（SP）＝60H，（PC）＝0123H，子程序SUBRTN的首地址为0456H。

执行指令：ACALL SUBRTN

机器码： 91 56

100 1	0 0 0 1
0101	0110

执行结果：

（PC）＋2＝0123H＋2＝0125H→（PC）

PC压栈：25H 压入(SP)＋1＝61H，01H压入(SP)＋l＝62H，此时（SP）＝62H。

构造目标地址：addr11 → PC10–0 PC=0456H

2）长调用指令

LCALL　　addr16　；PC←(PC)+3

；SP←(SP)+1，(SP)←$(PC)_{7\sim0}$

；SP←(SP)+1，(SP)←$(PC)_{15\sim8}$

；PC←addr16

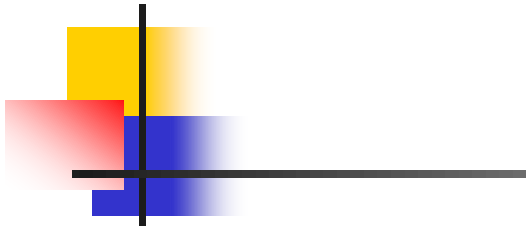
3）返回指令

RET　　；子程序返回指令

RETI　　；中断服务程序返回指令

$(PC)_{15\sim8}$←(SP)，SP←(SP)－1

$(PC)_{7\sim0}$←(SP)，SP←(SP)－1



- 所调用的子程序的首地址可以在64 KB范围内。

例：设（SP）＝60H，（PC）＝0123H，子程序SUBRTN的首地址为3456H。

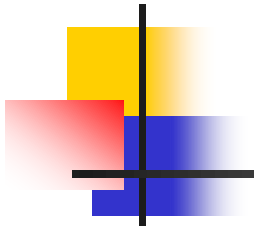
执行指令：LCALL SUBRTN

执行：

（PC）＋3＝0123H＋3＝0126H→（PC）。

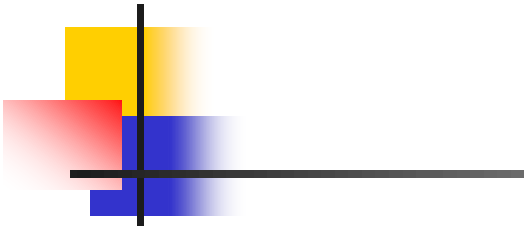
（PC）压栈：26H＝(6lH)，0lH＝(62H)，(SP)＝62H。

目标地址：(PC) = 3456H



R0的初值为05H，A的初值为02H，当在主程序中调用以下子程序后返回时，A和R0的内容?(递归)

```
SBR:    DJNZ  R0，NEXT
SBR1:   ADD   A，#02H
        RET
NEXT:   ACALL       SBR
        SJMP  SBR1
```



4. 空操作指令

NOP ； PC ← （PC）+1

3.6 布尔（位）操作类指令

概述：

- 布尔处理器实际上是一位的微处理机

  进位标志CY作为位累加器：

  内部RAM的位寻址区及部分SFR为位存储器：

  P0、P1、P2、P3为位I/O：

- 11种助记符，丰富的操作指令

- 寻址位的表示方法：

直接使用位地址	如：D5H
位名称	如：F0
单元地址加位	如：D0H.5
专用寄存器符号加位	如：PSW.5

** **详细参见寻址方式的位寻址**

1. 位传送指令

MOV C，bit；C← (bit)

MOV bit，C；bit ← (C)

- **无法对2个可寻址位直接进行传输 MOV bit, bit**
- **对I/O端口操作：读-修改-写**

指令先把端口的8位全读入，然后进行位传送，再把8位内容传送到端口的锁存器中。

例：设(C)=1，执行指令为 MOV P1．3，C，执行结果: P1.3=1

例：设P1口的内容为00111010B，执行指令如 MOV C，P1.3，执行结果为（C）＝1。



2. 布尔状态控制指令

- 清零：CLR　　C　　；C← 0 清零

  CLR　　bit　　；bit ← 0 清零

- 置位：SETB　　C　　；C←1

  SETB　　bit　　；bit ←1

- 取反：CPL　　C　　；C ←$\overline{(C)}$

  CPL　　bit　　；bit ←$\overline{(bit)}$

3. 布尔(位)逻辑操作指令

与，或，（取反）

ANL	C，bit	；C ←(C)∧$\overline{(bit)}$
ANL	C，/ bit	；C ←(C)∧(bit)
ORL	C，bit	；C ←(C)∨$\overline{(bit)}$
ORL	C，/ bit	；C ←(C)∨(bit)

没有位异或指令，用与或实现

$$D=E \oplus B=\bar{E}B+E\bar{B}$$

4. 布尔（位）条件转移指令

1）布尔累加器C 条件转移指令

- **（C）=1转移指令：**

JC　　rel　　；若（CY）=1，则PC ← （PC）+2+rel

　　；若（CY）≠1，则PC ← （PC）+2

- **（C）=0转移指令：**

JNC　rel　　；若（CY）= 0，则PC ← （PC）+2+rel

　　；若（CY）≠ 0，则PC ← （PC）+2

例10：温度控制程序

某温度控制系统，A中存温度采样值Ta，(20H)=温度下限值T20，(30H)=温度上限值T30。若Ta > T30，程序转降温JW，若Ta < T20，程序转升温SW，若T30≥Ta≥T20程序转FH返回主程序。

```
        CJNE  A,30H  LOOP
        AJMP  FH                    ；等于T30，转FH
LOOP:   JNC  JW                     ；大于T30，降温
        CJNE  A,20H,LOOP1
        AJMP  FH                    ；等于T20，转FH
LOOP1:  JC  SW                      ；小于T20，升温
FH:  ------                         ；保温
JW:  ------
SW : ------
```

2)．位测试条件转移指令

JB　　bit，rel　　;位状态为1转移

JNB　bit，rel　　;位状态为0转移

- 操作不影响标志位。
- 三字节指令

满足条件PC ← (PC)+3+rel;

不满足条件 PC ← （PC）+3

3)．位测试条件转移并清0指令

JBC bit, rel　;位状态为1转移，并使该位清0

- 操作不影响标志位。
- 三字节指令

满足条件PC ← (PC)+3+rel;

不满足条件 PC ← （PC）+3

例12：试编程序实现下述逻辑表达式的功能

$$Y=\overline{X0}+X1\overline{X2}+\overline{X1}X2+\overline{X4}\,\overline{X5}\,\overline{X6}X7$$

说明：设八位输入信号从P1口输入，Y信号从P3.0输出。

```
        MOV  A,P1
        JB  ACC.0,MM
        SETB  C
        SJMP  OUT          ；X0=1，转出口
MM：    MOV  C,ACC.1
        ANL  C,/ACC.2
        JC  OUT            ；X1X2=1，转出口
         MOV  C,ACC.2
        ANL  C,/ACC.1
        JC  OUT            ；X1X2=1，转出口
        MOV  C,ACC.7
        ANL  C,/ACC.4
        ANL  C,/ACC.5
        ANL  C,/ACC.6      ；X4X5X6X7
OUT：MOV  P3.0,C           ；输出结果
```

第三章总结

- 熟练掌握寻址方式：7种
- 掌握指令的执行过程
- 熟练掌握各类指令的用法及其格式

数据传送

算术运算

逻辑运算

控制转移

布尔（位）操作指令

- 作业：
- （12）（13）（14）（15）（16）
- （17）（20）